D1 Reading tracebacks and stack traces

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D1 Reading tracebacks and stack traces 3
D1-L1 Read a Python traceback from the bottom up 3
D1-L2 Spark errors, the condition and the real cause 8
D1-L3 Py4J and Spark Connect errors 12
Track D best-practice checklist 17
Appendix: solutions and answer keys 18

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D1 Reading tracebacks and stack traces

Roadmap: Every month. Find the root cause in Python and Spark errors.

D1-L1 Read a Python traceback from the bottom up

Concept

A traceback lists the calls that led to an error, oldest first. The line that matters most is at the **bottom**. Read it in this order:

1. **The last line** names the exception type and message: `NameError: name 'uuid' is not defined`. That is what went wrong.
2. **The frame just above it** shows where: file, line number, function, and the source line. Python 3.11 and later add `^` or `~` marks under the exact expression.
3. **Walk up** until you reach the first frame in **your own code**. Frames inside `site-packages` or the standard library are rarely the cause. The call from your code that fed them bad input usually is.

Here is your reference `log_pipeline_run`, cut down, on Python 3.13:

```
Traceback (most recent call last):
  File "observability.py", line 7, in <module>
    main()
    ~~~~^^
  File "observability.py", line 5, in main
    log_run()
    ~~~~~~~^^
  File "observability.py", line 2, in log_run
    return str(uuid.uuid4())
               ^^^^
NameError: name 'uuid' is not defined. Did you forget to import 'uuid'?
```

Python 3.12 added the "Did you forget to import" hint. Python 3.11 prints the same error without it.

Chained exceptions print two tracebacks. The joining sentence tells you how they relate:

- "**The above exception was the direct cause of the following exception**" means someone wrote `raise NewError(...) from err`. The first traceback is the root cause.
- "**During handling of the above exception, another exception occurred**" means a second error happened inside an `except` block. Your error handler has its own bug. Fix both.

When you catch an exception to add context, use `raise ... from err`. The original traceback stays attached, so the next person sees both.

Never read only the first line of a traceback, and never read only the message. `KeyError: 'row_count_max'` names the missing key. The frame tells you which dictionary, and so which config file, lacks it.

Why it matters

A contract validator failed with a 60-line traceback. The on-call engineer read the top frames, which pointed at the job scheduler, and restarted the job three times. The last line said `KeyError: 'row_count_max'`, and the frame above it named `_check_row_count`. The new tracker's contract file lacked one key. Reading from the bottom would have found it in a minute.

Worked example: weak code

```
def load_contract(path: str) -> dict[str, object]:
    """Load a contract file."""
    try:
        with open(path) as f:
            return json.load(f)
    except Exception:
        raise ValueError("bad contract")
```

1. **Line 6.** Catches everything, including a typo in your own code.
2. **Line 7.** No path, no cause. The traceback prints 'During handling...', and the reader must guess which error was the real one.

Worked example: strong code

```
import json


def load_contract(path: str) -> dict[str, object]:
    """Load a contract file.

    Raises:
        ValueError: If the file is not valid JSON.
    """
    try:
        with open(path, encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError as err:
        raise ValueError(f"contract {path} is not valid JSON: line
    {err.lineno}") from err
```

1. **Line 13.** Catch only the error you expect. A missing file still raises FileNotFoundError with its own traceback.

2. **Line 14.** The message names the file and line. `from err` keeps the original traceback as the direct cause.

Read the last line, then the frame above it, then walk up to your own code. When you re-raise, add context and chain with `from err`.

Recap card

- **Order for reading a traceback** Last line (what), frame above it (where), then up to the first frame in your own code.
- **'The above exception was the direct cause...'** Someone chained with raise ... from err. The first traceback is the root cause.
- **'During handling of the above exception...'** The except block raised a new error. Fix the handler and the original error.
- **Re-raising with context** Catch the narrow exception, add facts to the message, and chain with from err.

Practice

Exercise 1. Multiple choice (level 1)

A traceback is 40 lines long. Where do you start reading?

- a) The last line: exception type and message
- b) The first line after 'Traceback (most recent call last)'
- c) The longest frame
- d) The first frame inside site-packages

Why?

- r1) Frames run oldest first. The exception and the frame that raised it are at the bottom.
- r2) Python prints the most important frame last to save space.

Exercise 2. Spot the bug (level 2)

This traceback came from Python 3.13. Tap the frame line in **your own code** where you would start fixing. Your code is in `/repo/pipelines/`.

```
Traceback (most recent call last):
  File "/repo/pipelines/run_qa.py", line 31, in <module>
    report = check_weights(df, "final_weight")
  File "/repo/pipelines/qa_statistical.py", line 19, in check_weights
    "mean": round(stats["mean"], 4),
            ~~~~~^^^^^^^^^^^^^^^^^^
TypeError: type NoneType doesn't define __round__ method
```

Write down the line number of the bug.

Exercise 3. Multiple choice (level 2)

A traceback shows a `KeyError`, then the sentence "During handling of the above exception, another exception occurred", then a `NameError`. What does that tell you?

- a) The except block that handled the KeyError has its own bug
- b) The code used raise ... from err
- c) Only the NameError matters
- d) Python retried the code

Why?

- r1) The second exception was raised while handling the first, without an explicit chain. Both need a fix.
- r2) NameError always hides a KeyError.

Exercise 4. Write the function (level 3)

Write `first_own_frame(traceback_text: str, project_dir: str) -> tuple[str, int] | None`. Return the file path and line number of the **last** frame line (`File "<path>", line <n>, in <name>`) whose path starts with `project_dir`. That is the first frame in your code when you read from the bottom. Return `None` if no frame is in your code. For chained tracebacks, use the last traceback in the text.

Your code must pass these checks:

- Skip library frames and return the last frame inside project_dir, not the first.
- Return None when no frame is in your code.
- Use the last traceback in the text, and its last frame in your code.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
FRAME_PATTERN = ...

# Find the frame to start from when reading a traceback bottom up.
def first_own_frame(
    traceback_text: str,
    project_dir: str,
) -> tuple[str, int] | None: ...
```

Starter code:

```python
def first_own_frame(traceback_text, project_dir):
    for line in traceback_text.splitlines():
        if project_dir in line:
```

```
            return line
```

Exercise 5. Refactor (level 4)

Refactor `row_count_bounds` so a missing key raises `ValueError` naming the contract id and the missing key, chained to the original `KeyError` with `from`. Add type hints and a docstring.

Your code must pass these checks:

- Return (min, max) for a complete contract.
- Raise ValueError naming the contract and the key, with `from err` so the KeyError is the direct cause.
- Catch only KeyError. Other errors, such as passing None, must surface unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Row count bounds from a data contract.
def row_count_bounds(
    contract: dict[str, object],
) -> tuple[object, object]: ...
```

Starter code:

```
def row_count_bounds(contract):
    try:
        return contract["row_count_min"], contract["row_count_max"]
    except Exception:
        raise ValueError("bad contract")
```

Sources

- Python 3.11+ prints caret or tilde markers under the failing expression; 3.12 and 3.13 add 'Did you forget to import' to NameError for standard modules; 3.11 does not. (verified: Local runs on CPython 3.11.15, 3.12.3 and 3.13.12, September 2026.)
- raise ... from err prints 'The above exception was the direct cause of the following exception'; an error inside an except block prints 'During handling of the above exception, another exception occurred'. (verified: Local runs on CPython 3.11.15 and 3.13.12, September 2026.)
- round(None, 4) raises TypeError: type NoneType doesn't define **round** method. (verified: Local run on CPython 3.13.12, September 2026.)

D1-L2 Spark errors, the condition and the real cause

Concept

A Spark error message is long, but it has a fixed shape. In PySpark 4.0 (Databricks Runtime 17.3 LTS) most errors start with an **error condition** in square brackets and end with a **SQLSTATE**:

```
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or function
parameter with name `final_wieght` cannot be resolved. Did you mean one
of the following? [`final_weight`, `age`, `wave_id`, `respondent_id`].
SQLSTATE: 42703
```

Read three things:

- **The condition** names the problem. Search for it in the Databricks error conditions reference.
- **The SQLSTATE class**, its first two characters. Class **42** is a syntax error or access rule violation: your code names something that does not exist. Class **22** is a data exception: the code is valid but a value is not.
- **The origin line.** Runtime errors show `== DataFrame ==` and `"cast" was called from ...` with your file and line.

When the error appears. Transformations are lazy, so a data error appears at the **action** (`collect`, `count`, `write`), not at the line that caused it. The origin line points you back. In classic Spark, a missing column fails at once, when you call `select`.

Python UDF errors arrive as `PythonException` wrapping a normal Python traceback. Read that inner traceback bottom up (lesson D1-L1).

Shadowed built-ins give a misleading message. Your `data_contracts.py` imports `sum` from `pyspark.sql.functions`. Then `sum(1 for c in checks)` raises `[NOT_COLUMN_OR_STR] Argument col should be a Column or str, got generator`. The code never meant to call Spark. Import `functions as F`.

ANSI mode is on by default in Spark 4.0 and Databricks Runtime 17.0 and above. A cast of `'abc'` to `INT` now raises `[CAST_INVALID_INPUT]` instead of returning null. Use `try_cast` where bad values are expected, and count the nulls it produces.

In code, catch `pyspark.errors.AnalysisException` and read `err.getCondition()` and `err.getSqlState()`, not the message text.

Why it matters

After a runtime upgrade to Databricks Runtime 17.3 LTS, a silver job that had run for two years failed with `[CAST_INVALID_INPUT] The value 'n/a' ... cannot be cast to "INT"`. The engineer's first thought was a Spark bug. The SQLSTATE started with 22, a data exception: the data had always held 'n/a', and the old runtime had turned it into null without a word. The new error exposed a data quality problem that was two years old.

Worked example: weak code

```
1 from pyspark.sql.functions import col, sum
2
3 silver = bronze.withColumn("age", col("age").cast("int"))
4 passed = sum(1 for c in checks if c.status == "PASS")
5 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

1. **Line 1.** Shadows Python's sum. The next line fails with NOT_COLUMN_OR_STR.
2. **Line 3.** Under ANSI mode, one bad value fails the whole write, several lines later.

Worked example: strong code

```
1 from pyspark.sql import functions as F
2
3 silver = bronze.withColumn("age", F.col("age").try_cast("int"))
4 bad_ages = bronze.filter(F.col("age").isNotNull() &
      F.col("age").try_cast("int").isNull()).count()
5 passed = sum(1 for c in checks if c.status == "PASS")
6 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

1. **Line 1.** Python's sum stays Python's sum.
2. **Line 3.** Bad values become null on purpose.
3. **Line 4.** Then you count them, so nothing disappears silently.

Read the condition, the SQLSTATE class and the origin line. Class 42 means fix the code. Class 22 means look at the data.

Recap card

- **Shape of a Spark 4 error message** [ERROR_CONDITION] message ... SQLSTATE: xxxxx, often with an origin line under == DataFrame ==.
- **SQLSTATE class 42 vs 22** 42: syntax error or access rule violation, fix the code. 22: data exception, look at the data.

- **Why a data error appears at write()** Transformations are lazy. The action runs the plan. The origin line shows where the expression was written.
- **NOT_COLUMN_OR_STR ... got generator** Python's sum is shadowed by pyspark.sql.functions.sum. Import functions as F.

Practice

Exercise 1. Multiple choice (level 1)

A job fails with `[UNRESOLVED_COLUMN.WITH_SUGGESTION] ... name` final_wieght `cannot be resolved. Did you mean one of the following? [` final_weight `, ...]. SQLSTATE: 42703`. What do you fix?

- a) The column name in the code: final_wieght is a typo for final_weight
- b) The data: some rows lack final_weight
- c) The cluster: restart it
- d) The table permissions

Why?

- r1) Class 42 errors mean the code names something that does not exist. The message suggests the right name.
- r2) Spark always suggests the column you meant.

Exercise 2. Multiple choice (level 2)

`pass_rate` in your `data_contracts.py` runs `sum(1 for c in self.checks if c.status == "PASS")` and fails with `[NOT_COLUMN_OR_STR] Argument` col `should be a Column or str, got generator`. What is the cause?

- a) `sum` is PySpark's sum, imported by name, which shadows Python's built-in
- b) self.checks is a DataFrame
- c) Generators are not allowed in dataclasses
- d) The status column is missing

Why?

- r1) `from pyspark.sql.functions import sum` replaces the built-in in that module, so the generator goes to Spark's sum.
- r2) PySpark disables Python built-ins in notebooks.

Exercise 3. Spot the bug (level 3)

This job fails on line 5 with `[CAST_INVALID_INPUT] The value 'n/a' of the type "STRING" cannot be cast to "INT"` and `"cast" was called from ... line 3`. Tap the line that causes it, then fix it so bad ages become null. Keep `functions as F`.

```
1 from pyspark.sql import functions as F
2
3 silver = bronze.withColumn("age", F.col("age").cast("int"))
4 silver = silver.withColumn("wave_id", F.lit(wave_id))
5 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `triage(message: str) -> tuple[str | None, str]` for Spark error messages. Extract the error condition, the text in the first `[...]` at the start of the message, or `None`. Then classify by the SQLSTATE at the end (`SQLSTATE: 42703`): class `"42"` gives `"code"`, class `"22"` gives `"data"`, anything else or no SQLSTATE gives `"unknown"`.

Your code must pass these checks:

- Class 42 is a code error. Extract the condition inside the first brackets.
- Class 22 is a data exception.
- Read the class from the SQLSTATE, not from any '42' in the message.
- No brackets and no SQLSTATE: (None, 'unknown').

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1  # Module constant.
2  CONDITION_PATTERN = ...
3
4  # Module constant.
5  SQLSTATE_PATTERN = ...
6
7  # Module constant.
8  CLASS_LABELS = ...
9
10 # Error condition and a first-pass category for a Spark error message.
11 def triage(message: str) -> tuple[str | None, str]: ...
```

Starter code:

```
1 def triage(message):
2     if "42" in message:
3         return None, "code"
4     return None, "data"
```

Exercise 5. Architecture design (level 4)

Write the first-response steps for your team's runbook when a Spark job fails.

Options: The error condition, the SQLSTATE and the origin line, The Java stack trace from the top.

1. First thing to read
2. SQLSTATE class 22 after a runtime upgrade
3. Handling errors in code

Sources

- PySpark 4.0.4 error texts: UNRESOLVED_COLUMN.WITH_SUGGESTION with SQLSTATE 42703; CAST_INVALID_INPUT with SQLSTATE 22018 and an origin line; PythonException wrapping a Python traceback; NOT_COLUMN_OR_STR for a generator passed to shadowed sum; getCondition() and getSqlState() on AnalysisException; Column.try_cast exists. (verified: Local runs on PySpark 4.0.4, September 2026.)
- ANSI mode is enabled by default in Apache Spark 4.0 and Databricks Runtime 17.0 and above. (verified: https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance (checked 2026-09-25))
- SQLSTATE class 22 is data exception; class 42 is syntax error or access rule violation. (verified: https://docs.databricks.com/aws/en/error-messages/sqlstates (checked 2026-09-25))

D1-L3 Py4J and Spark Connect errors

Concept

PySpark talks to the Spark engine in one of two ways, and each shapes the errors you see. Facts below were checked on PySpark 4.0.4 and the Databricks docs in September 2026.

Spark Classic runs your Python next to a JVM driver through **Py4J**. PySpark converts most engine errors into Python classes such as `AnalysisException`. Some still arrive raw, as `py4j.protocol.Py4JJavaError`:

```
Py4JJavaError: An error occurred while calling
z:org.apache.spark.api.python.PythonRDD.collectAndServe.
: org.apache.spark.SparkException: Job aborted due to stage failure ...
Caused by: org.apache.spark.api.python.PythonException: Traceback ...
  File "/repo/job.py", line 5, in <lambda>
ZeroDivisionError: division by zero
```

Skip the Java frames. Search for the **last** `Caused by:` line and the Python traceback inside it. That is your bug.

Spark Connect sends a query plan to a remote server over gRPC. Databricks **serverless compute uses Spark Connect**, and supports only Spark Connect APIs. Three differences matter when you debug:

- **Analysis is deferred.** Classic Spark rejects a misspelt column when you call `select`. Spark Connect accepts the `select` and raises `AnalysisException` at the action, or when you read `df.columns` or `df.schema`. A `try` around the `select` catches nothing.
- **Error classes live in** `pyspark.errors.exceptions.connect`**.** They subclass `SparkConnectGrpcException` and the familiar base classes, so `except pyspark.errors.AnalysisException` still works. The message ends with a `JVM stacktrace:` section from the server.
- **JVM-only attributes fail.** `spark.sparkContext` raises `[JVM_ATTRIBUTE_NOT_SUPPORTED]`. RDD code, such as `sc.parallelize`, does not run on serverless.

Two habits follow. Catch `pyspark.errors` classes, never `Py4JJavaError`, so the same code works on both. And place error handling around the **action**, where both modes raise, not around the transformation.

Why it matters

A notebook wrapped `df.select(col(name))` in `try/except AnalysisException` to skip columns missing from older waves. It worked on a classic cluster. After the team moved the job to serverless, the `except` never fired: the `select` succeeded, and the `write` failed with the same `AnalysisException` two cells later. Checking `name in df.columns` before the `select` works in both modes.

Worked example: weak code

```
1 from py4j.protocol import Py4JJavaError
2 from pyspark.sql import functions as F
3
4 try:
2     selected = df.select(F.col(optional_column))
6 except Py4JJavaError:
7     selected = df
8 selected.write.mode("append").saveAsTable(target_table)
```

1. **Line 1.** Py4J exists only in Spark Classic. Serverless has no Py4J.
2. **Line 5.** Under Spark Connect this line never raises. The error arrives at the write.

Worked example: strong code

```
1  from pyspark.errors import AnalysisException
2  from pyspark.sql import functions as F
3
4  columns = [c for c in (key_column, optional_column) if c in df.columns]
5  selected = df.select(*[F.col(c) for c in columns])
6  try:
7      selected.write.mode("append").saveAsTable(target_table)
8  except AnalysisException as err:
9      raise RuntimeError(f"write to {target_table} failed:
       {err.getCondition()}") from err
```

1. **Line 4.** Check the schema explicitly. Works the same in Classic and Connect.
2. **Line 8.** A pyspark.errors class, caught around the action, where both modes raise.

Find the last "Caused by" in a Py4J error. Under Spark Connect, expect errors at the action, and catch pyspark.errors classes.

Recap card

- **Finding the cause in a Py4JJavaError** Search for the last 'Caused by:'. A Python UDF error holds a Python traceback there.
- **Spark Connect and missing columns** Analysis is deferred. The error comes at the action or when you read df.columns or df.schema.
- **Exception class that works on Classic and Connect** pyspark.errors.AnalysisException (and other pyspark.errors classes). Not Py4JJavaError.
- **Serverless compute and the Spark API** Spark Connect only. No RDD APIs. spark.sparkContext raises JVM_ATTRIBUTE_NOT_SUPPORTED.

Practice

Exercise 1. Multiple choice (level 1)

A raw `Py4JJavaError` fills 200 lines, mostly Java frames. Where is the cause of a Python UDF failure?

- a) After the last 'Caused by:', in the Python traceback it contains
- b) In the first Java frame
- c) In the line 'An error occurred while calling'
- d) It is not shown. You must read the executor logs

Why?

- r1) The JVM wraps the Python worker's exception. The innermost cause holds the Python traceback.

- r2) Java always prints the cause first.

Exercise 2. Predict the output (level 2)

This runs on **serverless** (Spark Connect). The DataFrame has no column `region`. What does it print?

```
from pyspark.errors import AnalysisException
from pyspark.sql import functions as F

try:
    out = df.select(F.col("region"))
    print("select ok")
except AnalysisException:
    print("caught")
```

Exercise 3. Multiple choice (level 2)

Which `except` clause catches a missing-column error on both a classic cluster and serverless?

- a) except pyspark.errors.AnalysisException
- b) except py4j.protocol.Py4JJavaError
- c) except pyspark.errors.exceptions.connect.SparkConnectGrpcException
- d) except KeyError

Why?

- r1) Both modes raise a subclass of pyspark.errors.AnalysisException.
- r2) AnalysisException is a Py4J class.

Exercise 4. Write the function (level 3)

Write `python_cause(message: str) -> str | None`. Given the text of a Py4J error, return the last line of the Python traceback in the **last** `Caused by: org.apache.spark.api.python.PythonException` section: the first non-blank line that does not start with whitespace, `Traceback` or `File`, after that marker. Return `None` if there is no such section.

Your code must pass these checks:

- Return the exception line after the PythonException marker, skipping frames and Java lines.
- Use the last PythonException section.
- No PythonException section: return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
```

```
MARKER = ...

# Python exception line inside the last PythonException of a Py4J error.
def python_cause(message: str) -> str | None: ...
```

Starter code:

```
def python_cause(message):
    return message.splitlines()[-1]
```

Exercise 5. Architecture design (level 4)

A notebook written for a classic cluster moves to serverless. Plan the changes to its error handling.

Options: Check `name in df.columns` before selecting, Wrap each select in try/except.

1. Optional columns in older waves
2. Exception class to catch
3. A helper that uses sc.parallelize

Sources

- Classic: raw Py4JJavaError for RDD Python failures with 'Caused by: org.apache.spark.api.python.PythonException' holding the Python traceback; eager analysis at select. Connect: analysis deferred to action or schema access; errors in pyspark.errors.exceptions.connect subclass SparkConnectGrpcException and are caught by pyspark.errors.AnalysisException; message includes 'JVM stacktrace:'; spark.sparkContext raises JVM_ATTRIBUTE_NOT_SUPPORTED. (verified: Local runs on PySpark 4.0.4, classic local[1] and remote('local[1]'), September 2026.)
- Serverless compute supports only Spark Connect APIs, not RDD APIs, and defers analysis and name resolution to execution time. (verified: https://docs.databricks.com/aws/en/compute/serverless/limitations and https://docs.databricks.com/aws/en/spark/connect-vs-classic (checked 2026-09-25))

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D1-L1 Read a Python traceback from the bottom up

D1-L1 exercise 1

Hints. (1) Read the header: 'most recent call last'. (2) Where is the most recent call? (3) At the bottom.

Answer. a) The last line: exception type and message Because: r1) Frames run oldest first. The exception and the frame that raised it are at the bottom.

The bottom holds what went wrong. The frames above say where and how you got there.

Common wrong answers.

- `b`: The first frame is the oldest call, often your entry point or a scheduler. The error happened at the other end.
- `c`: Length says nothing about relevance. Start at the bottom.
- `d`: Library frames rarely hold the cause. Find the last frame in your own code.
- `a|r2`: Right answer, wrong reason. The order is chronological: 'most recent call last'.

D1-L1 exercise 2

Hints. (1) Read the last line first. (2) Which frame is closest to the error and inside /repo/pipelines/? (3) The frame for qa_statistical.py, line 19.

Answer. The bug is on line 4.

`stats["mean"]` is None when the DataFrame is empty, and `round(None, 4)` raises TypeError. The frame for `qa_statistical.py` line 19 is where you add a guard.

Common wrong answers.

- Line 2: That is the entry point that called the function. The error happened deeper.
- Line 7: That line says what went wrong: round() received None. The frame above it says where.

D1-L1 exercise 3

Hints. (1) Which sentence does `from err` print? (2) This one is the other sentence. (3) The handler failed.

Answer. a) The except block that handled the KeyError has its own bug Because: r1) The second exception was raised while handling the first, without an explicit chain. Both need a fix.

"During handling" means your error handler raised. "Direct cause" means someone chained on purpose.

Common wrong answers.

- `b`: `from err` prints 'The above exception was the direct cause of the following exception'.
- `c`: The KeyError started it. Fix the handler and the missing key.
- `d`: Python does not retry. It records that the handler failed.
- `a|r2`: Right answer, wrong reason. The two errors are unrelated types. The handler raised a new one.

D1-L1 exercise 4

Hints. (1) Split on 'Traceback (most recent call last):' and keep the last part. (2) Match frame lines with a regular expression. (3) Keep overwriting the result, so the last match wins.

Answer.

```
import re

FRAME_PATTERN = re.compile(r'^\s*File "(?P<path>[^"]+)", line (?
  P<line>\d+)')


def first_own_frame(traceback_text: str, project_dir: str) -> tuple[str,
  int] | None:
    """Find the frame to start from when reading a traceback bottom up.

    Args:
        traceback_text: Full traceback text, possibly chained.
        project_dir: Path prefix of your own code.

    Returns:
        (path, line) of the last frame inside project_dir, or None.
    """
    last_block = traceback_text.split("Traceback (most recent call
  last):")[-1]
    found: tuple[str, int] | None = None
    for line in last_block.splitlines():
        match = FRAME_PATTERN.match(line)
```

```
        if match and match["path"].startswith(project_dir):
            found = (match["path"], int(match["line"]))
    return found
```

Reading bottom up is the same as keeping the last matching frame. Library frames never match your prefix.

D1-L1 exercise 5

Hints. (1) Catch `KeyError as err`, nothing wider. (2) `err.args[0]` is the missing key. (3) End the raise with `from err`.

Answer.

```
def row_count_bounds(contract: dict[str, object]) -> tuple[object, object]:
    """Row count bounds from a data contract.

    Args:
        contract: Parsed contract with contract_id, row_count_min and row_count_max.

    Returns:
        (row_count_min, row_count_max).

    Raises:
        ValueError: If a bound is missing.
    """
    try:
        return contract["row_count_min"], contract["row_count_max"]
    except KeyError as err:
        contract_id = contract.get("contract_id", "<unknown>")
        raise ValueError(f"contract {contract_id} is missing key {err.args[0]}") from err
```

A narrow except, a message with the facts, and an explicit chain. The next reader sees both tracebacks joined by "direct cause".

D1-L2 Spark errors, the condition and the real cause

D1-L2 exercise 1

Hints. (1) What does SQLSTATE class 42 mean? (2) Compare the name in the message with the suggestions. (3) A typo.

Answer. a) The column name in the code: final_wieght is a typo for final_weight Because: r1) Class 42 errors mean the code names something that does not exist. The message suggests the right name.

Unresolved names are code errors. Read the suggestion list, then fix the name.

Common wrong answers.

- `b`: Missing values are nulls, not an unresolved column. The name itself is wrong.
- `c`: The error is in the query plan. A restart runs the same plan.
- `d`: A permission error has a different condition. This one says the name cannot be resolved.
- `a|r2`: Right answer, wrong reason. The suggestions are the closest names, not always the right one. Check them.

D1-L2 exercise 2

Hints. (1) Which sum is being called? (2) Look at the module's imports. (3) Import functions as F.

Answer. a) `sum` is PySpark's sum, imported by name, which shadows Python's built-in Because: r1) `from pyspark.sql.functions import sum` replaces the built-in in that module, so the generator goes to Spark's sum.

Import `functions as F`. Python's `sum`, `min`, `max`, `round`, `abs` and `filter` then keep their meaning.

Common wrong answers.

- `b`: self.checks is a list of CheckResult objects. The function receiving it is the problem.
- `c`: Generators work anywhere. Look at the imports.
- `d`: No column is involved. The code never meant to call Spark.
- `a|r2`: Right answer, wrong reason. PySpark does not disable anything. The import replaced the name.

D1-L2 exercise 3

Hints. (1) Actions trigger the work. Where was the failing expression written? (2) The message names the origin: cast. (3) try_cast returns null instead of failing.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 silver = bronze.withColumn("age", F.col("age").try_cast("int"))
4 silver = silver.withColumn("wave_id", F.lit(wave_id))
5 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

The write triggers the plan, so the error surfaces there. The origin line points at the cast. Use `try_cast`, then count the nulls it creates.

Common wrong answers.

- Line 5: The error appears at the write because that is the action. The origin line says cast, on line 3.
- Line 4: Adding a literal cannot fail on 'n/a'.

D1-L2 exercise 4

Hints. (1) Match the condition with `^\s*\[([A-Z0-9_.]+)\]`. (2) Search for `SQLSTATE:\s*([0-9A-Z]{5})`. (3) The class is the first two characters of the SQLSTATE.

Answer.

```python
import re

CONDITION_PATTERN = re.compile(r"^\s*\[([A-Z0-9_.]+)\]")
SQLSTATE_PATTERN = re.compile(r"SQLSTATE:\s*([0-9A-Z]{5})")
CLASS_LABELS = {"42": "code", "22": "data"}


def triage(message: str) -> tuple[str | None, str]:
    """Error condition and a first-pass category for a Spark error message.

    Args:
        message: The error message text.

    Returns:
        (condition or None, "code", "data" or "unknown").
    """
    condition_match = CONDITION_PATTERN.match(message)
    condition = condition_match.group(1) if condition_match else None
    state_match = SQLSTATE_PATTERN.search(message)
    if state_match is None:
        return condition, "unknown"
    return condition, CLASS_LABELS.get(state_match.group(1)[:2], "unknown")
```

A first-pass triage sends class 42 to the code and class 22 to the data. Parse the structured parts, never loose substrings.

D1-L2 exercise 5

Hints. (1) Which part of the message is structured? (2) What did ANSI mode expose? (3) What stays stable across releases?

Answer.

- The condition and SQLSTATE give the category in seconds. The origin line gives the place.
- Class 22 means real bad data. Switching ANSI off hides it again.
- Conditions are stable identifiers. Message wording changes between releases.

Condition, SQLSTATE class, origin line. Then fix code or investigate data, depending on the class.

Common wrong answers.

- first = `java`: The JVM stack is Spark's internals. The condition and origin line point at your code.
- class22 = `ansi_off`: That returns to silent nulls. The bad values are still there, uncounted.
- catch = `text`: Message text changes between versions. Conditions do not.

D1-L3 Py4J and Spark Connect errors

D1-L3 exercise 1

Hints. (1) Wrapped exceptions nest. (2) Look for 'Caused by:'. (3) The last one.

Answer. a) After the last 'Caused by:', in the Python traceback it contains Because: r1) The JVM wraps the Python worker's exception. The innermost cause holds the Python traceback.

Search the message for the last "Caused by:" and read its Python traceback bottom up.

Common wrong answers.

- `b`: Java frames show Spark's internals, not your code.
- `c`: That line names the JVM method Python called, such as collectAndServe.
- `d`: The Python traceback is inside the message, after 'Caused by: org.apache.spark.api.python.PythonException'.
- `a|r2`: Right answer, wrong reason. The cause comes last, nested inside the outer exception.

D1-L3 exercise 2

Hints. (1) When does Spark Connect resolve column names? (2) At execution, or when you read the schema. (3) No action runs here.

Answer.

```
select ok
```

Spark Connect builds the plan without resolving names, so the `select` succeeds. `out.count()` or `out.columns` would raise.

Common wrong answers.

- `caught`: That is Spark Classic, which analyses eagerly. Spark Connect defers analysis to the action.

D1-L3 exercise 3

Hints. (1) Which package is shared by both modes? (2) pyspark.errors. (3) AnalysisException.

Answer. a) except pyspark.errors.AnalysisException Because: r1) Both modes raise a subclass of pyspark.errors.AnalysisException.

Catch the base classes in `pyspark.errors`. Your code then works in both modes.

Common wrong answers.

- `b`: Py4J exists only in Spark Classic, and PySpark converts most errors before you see them.
- `c`: That class exists only under Spark Connect.
- `d`: Column errors are not Python KeyErrors.
- `a|r2`: Right answer, wrong reason. It is a pyspark.errors class, with Classic and Connect subclasses.

D1-L3 exercise 4

Hints. (1) `message.rsplit(MARKER, 1)[1]` gives the text after the last marker. (2) Skip the rest of the marker line, then indented lines. (3) Java frames start with a tab, which is whitespace.

Answer.

```
MARKER = "Caused by: org.apache.spark.api.python.PythonException:"


def python_cause(message: str) -> str | None:
    """Python exception line inside the last PythonException of a Py4J
    error.

```

```
    Args:
        message: Full error text.

    Returns:
        The exception line, such as "ZeroDivisionError: division by zero",
    or None.
    """
    if MARKER not in message:
        return None
    section = message.rsplit(MARKER, 1)[1]
    for line in section.splitlines()[1:]:
        if not line.strip() or line[0].isspace():
            continue
        if line.startswith(("Traceback", "File")):
            continue
        return line
    return None
```

The innermost cause is last. `rsplit` with a limit of 1 finds it directly.

D1-L3 exercise 5

Hints. (1) When does Connect resolve names? (2) Which package exists in both modes? (3) What does serverless support?

Answer.

- Spark Connect defers analysis, so a try around a select catches nothing. An explicit schema check works in both modes.
- pyspark.errors classes exist in both modes. Py4J exists only in Classic.
- Serverless supports only Spark Connect APIs, not RDDs.

Explicit schema checks, pyspark.errors classes, and the DataFrame API only.

Common wrong answers.

- optional = `try_select`: Under Spark Connect the select succeeds and the error moves to the action.
- except_class = `py4j`: Serverless has no Py4J.
- rdd = `keep`: Spark RDD APIs are not supported on serverless compute.